

LECTURE NOTES

Algorithm and Programming

Introduction to Algorithm and Programming

Pandu Dwi Luhur Pambudi, S.Kom., M.Kom., M.I.M



Learning Outcomes

1. LO1: Menerapkan sintaks dan fungsi dalam bahasa C untuk pemecahan masalah.

Outline Materi

1. Definisi Algoritma
2. Representasi algoritma
3. Teorema Struktur
4. Struktur C

1.1 Definisi Algoritma

Secara etimologis, Algoritma berasal dari kata Al Khwarizmi / algorism. Dimana kata algorism digunakan untuk proses perhitungan aritmatika dengan menggunakan bahasa Arab. Tetapi, dalam pengertian lain, algoritma adalah urutan langkah demi langkah yang logis untuk menyelesaikan masalah yang disusun secara sistematis.

Tidak semua prosedur dapat disebut algoritma. Algoritma harus memiliki karakteristik berikut

- **Unambiguous**, Algoritma harus jelas dan tidak ambigu. Masing-masing langkahnya (atau fase), dan input/outputnya harus jelas dan hanya harus mengarah pada satu makna.
- **Input**, Algoritma harus memiliki 0 atau lebih input yang terdefinisi dengan baik.
- **Output**, Algoritma harus memiliki 1 atau lebih output yang terdefinisi dengan baik dan harus sesuai dengan output yang diinginkan.
- **Finiteness**, Algoritma harus berakhir setelah jumlah langkah yang terbatas
- **Feasibility**, Harus layak dengan sumber daya yang tersedia
- **Independent**, Suatu algoritma harus memiliki arahan langkah demi langkah, yang harus independen dari kode pemrograman apa pun.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Bagaimana sejarah kata **algoritma** dapat mempengaruhi cara kita memahami konsep dasar algoritma di era modern? Diskusikan relevansi asal-usul istilah ini dengan cara kita mendesain solusi komputasi hari ini.

Hint:

- Bahas siapa **Al Khwarizmi** dan kontribusinya dalam matematika.
- Hubungkan penggunaan **algorism** pada perhitungan aritmatika kuno dengan **algoritma modern** pada problem-solving.
- Diskusikan: meski istilahnya dari Arab dan perhitungan angka, mengapa sekarang cakupannya jadi lebih luas, tidak hanya aritmatika.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Dari enam karakteristik algoritma (Unambiguous, Input, Output, Finiteness, Feasibility, Independent), menurut Anda, karakteristik mana yang paling sulit dipenuhi dalam merancang algoritma nyata? Berikan contoh kasusnya.

Hint:

- Arahkan diskusi ke contoh nyata: misalnya **Infinite loop**, **ambiguitas instruksi**, atau algoritma yang tidak feasible di hardware terbatas.
- Bandingkan: menulis algoritma di kertas vs. implementasi di sistem riil.
- Ajak mahasiswa mendebat: mana yang lebih menantang, kejelasan langkah atau finiteness?

Pertanyaan Diskusi:

Mengapa sebuah algoritma harus **independen dari bahasa pemrograman**? Jelaskan dengan contoh bagaimana algoritma yang sama dapat diterapkan di berbagai bahasa tanpa mengubah logikanya.

Hint:

- Diskusikan contoh sederhana: algoritma bubble sort, pseudocode vs. C implementation.
- Tanyakan: kenapa jika algoritma terlalu terikat syntax, akan sulit dipahami lintas platform.
- Bahas pentingnya **representasi algoritma** dalam flowchart atau pseudocode sebelum coding.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (What exactly is an algorithm? Algorithms explained: [YouTube](#))

Belajar Mandiri



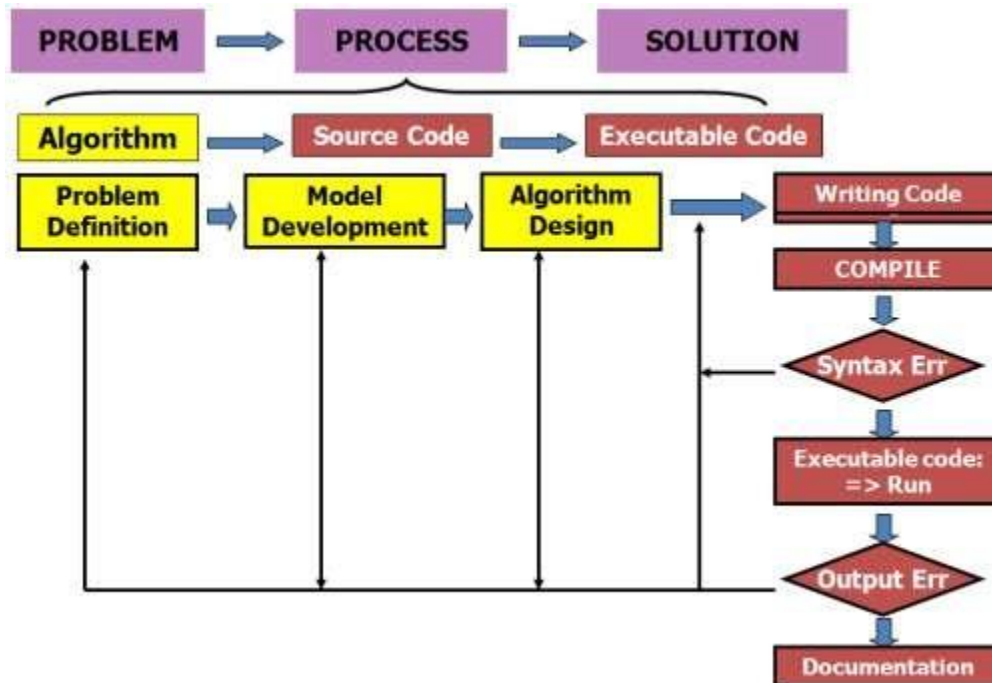
Pelajari secara mandiri terkait subtopik ini pada:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms*. MIT press.

1.2

Representasi Algoritma

Dalam menyelesaikan suatu permasalahan dengan menggunakan algoritma tertentu hingga menjadi suatu program. Berikut gambaran tahapan algoritma hingga menjadi program :



Terdapat langkah-langkah tahapan yang dapat diterapkan.

- Mendefinisikan masalah: Mendefinisikan masalah yang ingin dipecahkan sehingga jelas lingkup proses dari permasalahan yang ada
- Membuat model: Membuat model / bentuk matematis yang dapat digunakan dalam memecahkan masalah, misalnya apakah diperlukan pengurutan terhadap data, atau memerlukan perhitungan khusus yang diperlukan.
- Merancang Algoritma: Merancang algoritma adalah hal-hal yang perlu dipertimbangkan seperti masukkan (input), cara jalan proses dalam menyelesaikan permasalahan (process), dan hasil keluaran yang diharapkan (output).
- Menulis program: Menulis program adalah mengubah hasil algoritma yang telah dibentuk menjadi program (source code) sesuai dengan bahasa pemrograman tertentu.
- Executablecode: Melakukan compilesource code menjadi executable code. Kemudian memeriksa hasil compile, jika terjadi kesalahan maka kembali ke tahap sebelumnya (tahap 4 menulis program).

- Menjalankan program (run): Menjalankan program dilakukan untuk menguji kebenaran dengan menggunakan data input yang telah ditentukan untuk menghasilkan output yang diharapkan.
- Memperbaiki kesalahan (fix error): Jika dalam menjalankan program hasil keluaran (output) yang diharapkan tidak sesuai maka, kemungkinan kesalahan terjadi saat merancang algoritma, atau konversi algoritma menjadi source code, atau salah dalam menentukan model, kesalahan juga dapat terjadi pada saat mendefinisikan masalah. Hal ini dapat diselesaikan dengan mengulangi langkah yang sesuai dengan permasalahan yang dihadapi.
- Dokumentasi: Melakukan dokumentasi dari program yang sudah benar.

A. Pseudo-code

Pseudo-code adalah kode-kode yang dibuat agar dapat dimengerti yang selanjutnya diolah kembali dan diubah ke dalam suatu bahasa pemrograman. Selain itu pseudo-code dapat diartikan juga sebagai cara untuk menuliskan sebuah algoritma secara high-level (level tingkat tinggi).

Dalam penulisan pseudo-code biasanya dituliskan dengan kombinasi bahasa yang mudah dimengerti (bahasa manusia) dan notasi matematika dan biasanya sebuah pseudo-code tidak terlalu detail jika dibandingkan dengan kode pemrograman (source code).

Berikut contoh dari pseudo-code

Contoh pseudo-code untuk meminta inputan nama dan mencetaknya :

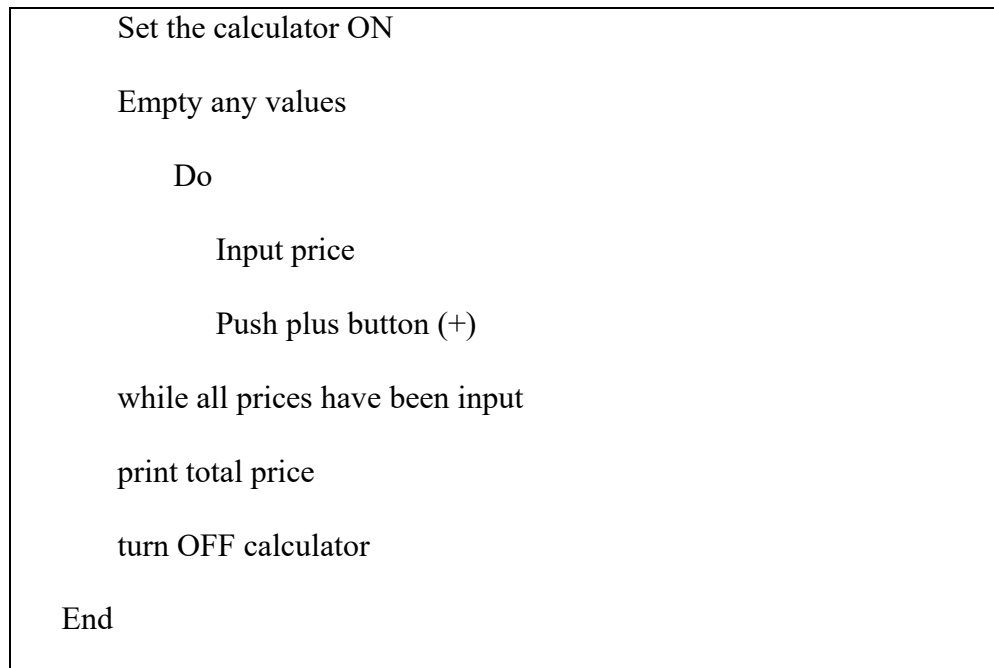
- Read nama
- Print nama

Dalam bahasa C pseudo-code tersebut dapat diubah menjadi :

```
char nama[100];  
  
scanf("%s", nama);      -> read nama  
  
printf("nama : %s", nama); -> print nama
```

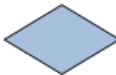
Contoh: Algoritma Menggunakan Kalkulator Untuk Menjumlahkan Nilai

Start



B. Flow Chart

Flow chart merupakan penggambaran secara grafik mengenai langkah-langkah dalam menyelesaikan suatu masalah suatu program. Flow chart juga biasa disebut sebagai diagram alir yang terdiri dari lambang-lambang fungsi tertentu.

Symbol	Function
	Terminator: START, END
	Process
	Selection
	Document
	Route
	Connector
	Connector between pages

Flow chart selalu diawali dan diakhiri dengan lambang terminator. Aliran yang ada pada flow chart selalu mulai dari atas ke bawah dan dari kiri ke kanan, langkah demi langkah. Dalam flow chart tidak ada proses yang dikerjakan secara bersamaan secara sekaligus, sehingga flow chart ini selalu dikerjakan satu persatu.

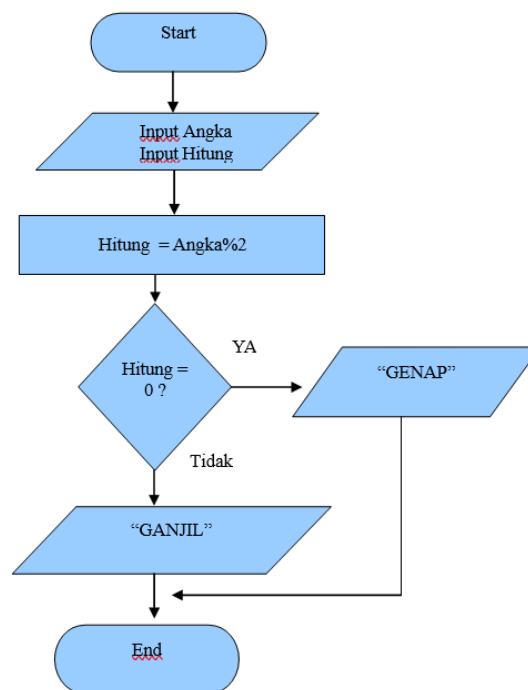
Flow chart ini merupakan penggambaran yang dibuat untuk mengidentifikasi proses dari permasalahan yang akan diselesaikan. Terdapat 3 proses yang dibuat untuk menyelesaikan permasalahan : input, process, dan output.

Sebagai contoh : langkah-langkah dalam menentukan bahwa suatu bilangan tersebut adalah bilangan ganjil atau genap?

Input : bilangan bulat.

Proses : menentukan berdasarkan inputan apakah bilangan yang dimasukkan merupakan bilangan ganjil atau genap. Hal ini dilakukan dengan mendapatkan hasil sisa bagi dari input bilangan yang dibagi dengan 2. Jika hasil sisa bagi 0 maka bilangan tersebut merupakan bilangan genap, jika hasil sisa bagi adalah 1 maka bilangan tersebut merupakan bilangan ganjil.

Output : Mencetak bilangan Ganjil atau bilangan Genap. Berikut gambaran dari flow chart berdasarkan soal tersebut :



Dari penggambaran flow chart tersebut dapat terlihat bagaimana penggambaran cara jalan dari code pemrograman yang akan dirancang nantinya.

Terdapat juga langkah-langkah yang dapat digunakan untuk membuat pemrograman komputer, dapat dibagi menjadi 3 kelompok proses :

- Sequence process: Pada sequence proses seluruh instruksi dikerjakan sesuai dengan satu demi satu sesuai dengan susunan dari instruksi pertama hingga instruksi terakhir. Instruksi kedua akan dikerjakan setelah instruksi pertama selesai dikerjakan.

- Selection process: Selection proses adakalanya suatu instruksi akan boleh dikerjakan jika memenuhi persyaratan yang telah ditentukan sebelumnya. C menyediakan tiga jenis struktur seleksi dalam bentuk pernyataan, antara lain : if single-selection statement, if...else double-selection statement dan switch multiple-selection statement. Misal jika terdapat suatu transaksi pembayaran dengan dua metode. Metode pertama secara tunai yang diberikan diskon 15%, dan metode kedua secara kredit dan tidak diberikan diskon. Dalam hal ini transaksi pembayaran hanya boleh melakukan 1 metode namun memiliki 2 alternatif pembayaran (diskon = 15% atau diskon =0).
- Iteration process: Pada iteration proses adakalanya suatu instruksi dapat dikerjakan berulang-ulang selama suatu kondisi terpenuhi. Misalnya jika suatu instruksi meminta inputan dari pengguna program. C menyediakan tiga jenis struktur iterasi dalam bentuk pernyataan, yaitu while, do...while, dan for. Pernyataan ini melakukan tugas berulang kali. Sebagai contoh diberikan nilai awal 0 dan nilai akhir 50 (0-50 adalah batasan nilai). Instruksi ini akan dikerjakan berulang selama nilai yang dimasukkan belum mencapai nilai akhir (50).

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Mengapa pseudo-code penting dibuat sebelum menulis source code? Diskusikan kelebihan pseudo-code dibanding langsung menulis program di bahasa C.

Hint:

- Arahkan ke fungsi pseudo-code sebagai *jembatan* antara logika manusia dan bahasa mesin.
- Bahas: pseudo-code bebas syntax error karena ditulis high-level.
- Minta contoh: apa resiko jika tidak buat pseudo-code, langsung coding? (contoh: salah alur, logika kabur, debugging lebih sulit).

Pertanyaan Diskusi:

Flowchart dan pseudo-code sama-sama merepresentasikan algoritma. Menurut Anda, dalam kondisi apa lebih baik menggunakan flowchart, dan kapan pseudo-code lebih bermanfaat?

Hint:

- Ajak membandingkan visual vs. tekstual: flowchart mudah dilihat untuk diskusi tim non-programmer; pseudo-code cocok untuk programmer.
- Diskusikan contoh: untuk algoritma pendek (misal ganjil-genap) vs. algoritma kompleks (misal sistem kasir diskon).
- Tanyakan: apakah flowchart wajib diubah ke pseudo-code? Mengapa?

Pertanyaan Diskusi:

Bagaimana perbedaan sequence, selection, dan iteration dapat memengaruhi cara kita merancang algoritma? Diskusikan contoh nyata di kehidupan sehari-hari.

Hint:

- Ajak mahasiswa beri contoh:
 - **Sequence:** prosedur resep memasak → tahap demi tahap.
 - **Selection:** menyalakan AC hanya jika suhu ruangan panas.
 - **Iteration:** alarm snooze → bunyi berulang hingga dimatikan.
- Diskusikan: jika salah memilih struktur kontrol, apa dampaknya pada program? (misal infinite loop, logika salah).

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Algorithm Vs Flowchart Vs Pseudocode: [YouTube](#))

Belajar Mandiri



Pelajari secara mandiri terkait subtopik ini pada:

- Tarsini, I., & Anggraeni, R. (2024). Explore flowchart and pseudocode concepts in algorithms and programming. *Indonesian Journal of Multidisciplinary Science*, 3(5).

1.3 Teorema Struktur

A. Sejarah bahasa C (History of C)

Bahasa C dirancang oleh Dennis M. Ritchie di Bell Laboratories pada tahun 1972. Bahasa C dikembangkan dari bahasa BCPL dan bahasa B. Bahasa BCPL dikembangkan oleh Martin Richards pada tahun 1967 sebagai bahasa untuk menulis sistem operasi dan compiler. Pada tahun 1970 Ken Thompson merancang bahasa B dengan memasukkan feature BCPL. Bahasa B dirancang dengan tujuan untuk digunakan membuat sistem operasi UNIX untuk komputer DEC PDP-7 pada Bahasa C dirancang oleh Dennis M. Ritchie di Bell Laboratories pada tahun 1972. Bahasa C dikembangkan dari bahasa BCPL dan bahasa B. Bahasa BCPL dikembangkan oleh Martin Richards pada tahun 1967 sebagai bahasa untuk menulis sistem operasi dan compiler. Pada tahun 1970 Ken Thompson merancang bahasa B dengan memasukkan feature BCPL. Bahasa B dirancang dengan tujuan untuk digunakan membuat sistem operasi UNIX untuk komputer DEC PDP-7 pada Bell Laboratories. Pada tahun 1978 Dennis M. Ritchie dan Brian W. Kernighan mempublikasikan bahasa C.

Bahasa pemrograman C merupakan bahasa pemrograman yang bersifat umum, tidak dikhususkan untuk bidang aplikasi tertentu. Bahasa pemrograman ini digolongkan sebagai bahasa pemrograman tingkat menengah (medium-level language). Hal ini dikarenakan bahasa C memiliki kemampuan dalam mengakses mesin komputer yang mendekati kemampuan bahasa rakitan, tetapi mudah dipelajari dan digunakan seperti halnya bahasa pemrograman tingkat tinggi. Selain itu, bahasa C memiliki karakteristik lain seperti : hemat ekspresi, alur kontrol, menggunakan struktur data modern, dan kaya dengan operator. Bahasa C merupakan bahasa pemrograman yang banyak digunakan mulai dari komputer mikro hingga super computer. Sampai saat ini bahasa C merupakan bahasa pemrograman yang paling banyak digunakan untuk membuat sistem operasi dan sistem software lainnya.

B. Standard library dalam C

Di C, Anda menggunakan fungsi untuk memodulasi program dengan menggabungkan fungsi baru yang Anda tulis dengan fungsi library standar C yang dikemas. Library standar C menyediakan koleksi fungsi yang kaya untuk melakukan perhitungan matematika umum, manipulasi string, manipulasi karakter, input/output, dan banyak operasi bermanfaat lainnya. Fungsi yang dikemas membuat pekerjaan Anda lebih mudah karena mereka menyediakan banyak kemampuan yang Anda butuhkan.

Standar C mencakup bahasa C dan library standarnya - kompilernya standar mengimplementasikan keduanya. Biasakan diri anda dengan koleksi yang kaya dari fungsi library standar C untuk membantu mengurangi waktu pengembangan program dan dengan menggunakan fungsi di library standar C membantu membuat program lebih portabel.

Setiap library standar memiliki header yang sesuai yang berisi prototipe fungsi untuk semua fungsi di library dan definisi berbagai jenis data dan konstanta yang dibutuhkan oleh fungsi tersebut. Standar C mencakup header tambahan.

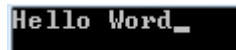
Contoh Fungsi library Standar C

No	Library	Deskripsi
1	<math.h>	Berisi Fungsi Prototipe untuk Fungsi Perpustakaan Matematika.
2	<stdio.h>	Berisi fungsi prototipe untuk fungsi dan informasi pustaka input/output standar yang digunakan oleh mereka.
3	<stdlib.h>	Berisi fungsi prototipe untuk konversi angka ke teks dan teks ke angka, alokasi memori, angka acak dan fungsi utilitas lainnya
4	<string.h>	Berisi fungsi prototipe untuk fungsi pemrosesan string
5	<time.h>	Berisi fungsi prototipe dan jenis untuk memanipulasi waktu dan tanggal

Perhatikan contoh berikut ini :

```
#include <stdio.h>
int main()
{
    printf("Hello Word");

    getchar();
    return 0;
}
```



Pada contoh tersebut fungsi printf akan jalan jika standart library<stdio.h> tersedia.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Bagaimana pengaruh evolusi dari BCPL $\rightarrow$ B $\rightarrow$ C memengaruhi kemampuan Bahasa C hingga menjadi bahasa populer untuk sistem operasi dan aplikasi modern?

Hint:

- Bahas tujuan awal: BCPL dibuat untuk compiler, B dibuat untuk sistem operasi UNIX.
- Bahasa C menyempurnakan fitur B & BCPL: closer to machine + easy syntax.
- Diskusikan: kenapa arsitektur low-level + high-level menjadikan C serbaguna di microcontroller hingga supercomputer.

Pertanyaan Diskusi:

Teorema Struktur menjelaskan bahwa setiap program dapat dibangun dengan tiga struktur dasar: sequence, selection, dan iteration. Diskusikan bagaimana bahasa C memfasilitasi ketiga struktur ini dengan contoh sederhana!

Hint:

- **Sequence:** instruksi dieksekusi baris per baris.
- **Selection:** if, if-else, switch-case.
- **Iteration:** for, while, do-while.
- Minta mahasiswa kasih contoh mini: misalnya hitung bilangan genap dengan selection, print 1–10 dengan iteration.

Pertanyaan Diskusi:

Library standar seperti `<stdio.h>`, `<math.h>`, `<string.h>` mempermudah pembuatan program. Menurut Anda, apa keuntungan dan resiko jika seorang programmer terlalu mengandalkan library standar?

Hint:

- **Keuntungan:** hemat waktu, fungsi sudah diuji, portable di berbagai compiler.
- **Resiko:** programmer bisa kurang paham logika internal, debugging jadi susah kalau fungsi library error, keterbatasan fitur untuk kasus sangat spesifik.
- Ajak diskusi: kapan lebih baik buat fungsi custom sendiri?

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Struktur Bahasa Programming menggunakan C: [YouTube](#))

1.4

Struktur C

Bahasa C merupakan bahasa pemrograman yang struktural hal ini dikarenakan:

Bahasa pemrograman C hanya memiliki 1 fungsi utama (fungsi main) dimana ketika program tersebut di compile, maka fungsi utama yang akan dijalankan pertama kali.

Bahasa pemrograman C sangat case sensitif. Maksudnya adalah huruf besar dan kecil sangat diperhatikan dalam pemrograman ini.

Misal :

- `printf("Hello Word");`
- `Printf("Hello Word");`

Antara statement 1 dan statement 2 terlihat perbedaanya yaitu ketika fungsi `printf` dengan `Printf` maka statement 2 akan tidak dikenali dalam bahasa C. Namun untuk statement 1 akan dikenali di bahasa C

Setiap pernyataan pada bahasa C harus diakhiri dengan ; (semi-colon). Misal :

- `printf("Algoritma");`
- `printf("Algoritma")`

Antara statement 1 dan statement 2 terlihat perbedaanya yaitu ketika statement 1 diakhiri dengan ; (semi-colon) maka statement1 akan dalam bahasa C. Namun untuk statement2tidak akan dikenali di bahasa C, dikarenakan statement 2 tidak diakhiri dengan ; (semi-colon).

Program C pada dasarnya memiliki formulir berikut:

- Perintah Preprocessor
- Ketik Definisi
- Fungsi Prototipe - Deklarasikan jenis fungsi dan variabel yang diteruskan ke fungsi.
- Variabel
- Fungsi
- Kita harus memiliki fungsi main ().

Kita harus memiliki fungsi `main()`.

Fungsi memiliki formulir:

`type function_name (parameters)`

```
{
    local variables
    C Statements
}
```

Jika definisi tipe dihilangkan C mengasumsikan bahwa fungsi mengembalikan tipe integer. Catatan: Ini bisa menjadi sumber masalah dalam suatu program.

A. Comment

Satu -satunya tujuan memasukkan komentar dalam program C Anda adalah untuk membantu Anda mendokumentasikan apa yang dilakukan program atau beberapa bagian tertentu dalam

program. Ingat, komentar ditulis untuk programmer seperti Anda. Misalnya, ketika Anda membaca kode seseorang, komentar dalam kode membantu Anda memahami apa yang dilakukan kode, atau setidaknya apa yang ingin dilakukan kode.

Anda tidak perlu khawatir tentang ukuran atau kecepatan kinerja program C Anda jika Anda menambahkan banyak komentar untuk itu. Menambahkan komentar ke dalam program C tidak meningkatkan ukuran kode biner program (yaitu, file yang dapat dieksekusi), meskipun ukuran program itu sendiri (yaitu, kode sumber) dapat menjadi lebih besar. Juga, kecepatan kinerja file yang dapat dieksekusi yang dibuat dari program C Anda tidak terpengaruh oleh komentar di dalam program C Anda.

Anda dapat menggunakan tanda komentar pembuka, `/*`, dan menutup tanda komentar, `*/`, untuk membantu Anda menguji dan memperbaiki kesalahan yang ditemukan dalam program C Anda. Anda dapat mengomentari satu atau lebih pernyataan C dalam program C Anda dengan `/*` dan `*/` ketika Anda perlu fokus pada pernyataan lain dan menonton perilaku mereka dengan cermat. Kompiler C akan mengabaikan pernyataan yang Anda komentari.

Kemudian, Anda selalu dapat mengembalikan pernyataan yang sebelumnya dikomentari hanya dengan menghapus komentar pembuka dan menutup tanda komentar. Dengan cara ini, Anda tidak perlu menghapus atau menulis ulang pernyataan apa pun selama pengujian dan debugging.

Contoh

```
/*-----
    My First Program
    -----*/

#include<stdio.h>

void main() {
    printf ("Hello, BINUSIAN\n");
}
```

B. Escape sequences

Berbagai karakter kontrol, seperti Newline dan Tab, harus diwakili oleh Escape sequence. Escape sequence diwakili oleh backslash (`\`), diikuti oleh karakter sequence tertentu. Tabel berikut mencantumkan escape sequences dan tindakan yang mereka sebabkan.

No	Escape sequence	Deskripsi
1	<code>\a</code>	bell, alert, system beep
2	<code>\b</code>	back space
3	<code>\t</code>	horizontal tab
4	<code>\n</code>	new line, line feed
5	<code>\v</code>	vertical tab
6	<code>\r</code>	carriage return
7	<code>\'</code>	single quote
8	<code>\"</code>	double quote
9	<code>\\</code>	backslash
10	<code>\xdd</code>	hexadecimal notation
11	<code>\ddd</code>	octal notation

Escape Sequence	Description	Unicode
<code>\t</code>	(Horizontal) Tab	<code>\u0009</code>
<code>\f</code>	Form Feed	<code>\u000c</code>
<code>\v</code>	Vertical Tab	<code>\u000b</code>
<code>\n</code>	New Line Feed (LF)	<code>\u000a</code>
<code>\r</code>	Carriage Return (CR)	<code>\u000d</code>
<code>\b</code>	Backspace	<code>\u0008</code>
<code>\'</code>	Single quote (grave accent)	<code>\u0027</code>
<code>\\</code>	Back slash (solidus)	<code>\u005c</code>
<code>\0</code>	null (C string terminator)	<code>\u0000</code>
<code>\a</code>	Alert (beep sound) or New line break for CSS2 content	<code>\u0007</code>
<code>\c</code>	continuation (UNIX)	Tidak ada
<code>\"</code>	Double quote	<code>\u0022</code>

Perhatikan contoh berikut ini

```
#include <stdio.h>
int main()
{
    printf("Word 1\t:\t\"C is simple!\"\n");
    printf("Word 2\t:\t\"Why?\"\n");
    printf("Word 1\t:\t\"Because it's easy to learn\"\n");
    printf("Word 2\t:\t\"Cool!!!\"\n");
    getchar();
    return 0;
}
```

```
Word 1 :      "C is simple!"
Word 2 :      "Why?"
Word 1 :      "Because it's easy to learn"
Word 2 :      "Cool!!!"
```

C. Character

Teks biasanya disimpan sebagai array karakter, seperti yang kami lakukan dengan garis [] dalam contoh di atas. Dengan konvensi dalam C, karakter terakhir dalam array karakter harus `\0` 'karena sebagian besar program yang memanipulasi array karakter mengharapkannya. Misalnya, Printf menggunakan `\0` 'untuk mendeteksi akhir array karakter saat mencetaknya dengan `%'s`.

Set pemindaian memindai karakter dalam aliran input, hanya mencari karakter yang cocok dengan karakter yang terkandung dalam set pemindaian. Setiap kali karakter dicocokkan, disimpan dalam argumen array karakter yang sesuai dengan pemindaian.

Bahasa program C ditulis dengan menggunakan sebagian dari karakter ASCII (American Standard Code for Information Interchange), yaitu:

- Huruf besar -> A B C sampai Z.
- Huruf kecil -> a b c sampai z.
- Angka -> 0 1 2 sampai 9
- Karakter khusus -> `„!“, „&“, „+“, „\“, „_“,` dan lain-lainnya

D. Identifier

Nama variabel dapat berupa pengidentifikasi yang valid. Setiap pengidentifikasi dapat terdiri dari huruf, digit, dan garis bawah (`_`), tetapi mungkin tidak dimulai dengan digit. C case sensitif, jadi `A1` dan `a1` adalah pengidentifikasi yang berbeda. Nama variabel harus dimulai dengan huruf kecil. Nanti dalam teks, kami akan memberikan signifikansi khusus untuk pengidentifikasi yang dimulai dengan huruf kapital dan pengidentifikasi yang menggunakan semua huruf kapital.

Memilih nama variabel yang bermakna membantu membuat program-pendorong diri, sehingga lebih sedikit komentar yang dibutuhkan. Hindari memulai pengidentifikasi dengan garis bawah (`_`) untuk mencegah konflik dengan pengidentifikasi yang dihasilkan kompiler dan pengidentifikasi perpustakaan standar. Nama variabel multi-kata dapat membuat program lebih mudah dibaca. Untuk nama -nama seperti itu:

- Pisahkan kata -kata dengan garis bawah, seperti dalam `total_commissions`, atau
- Jalankan kata -kata bersama dan mulailah setiap kata berikutnya dengan huruf kapital, seperti dalam `total komisi`.

Gaya terakhir disebut camel casing karena pola huruf besar dan huruf kecil menyerupai siluet camel. Kami lebih suka camel casing.

E. Identifier

Nama variabel dapat berupa pengidentifikasi yang valid. Setiap pengidentifikasi dapat terdiri dari huruf, digit, dan garis bawah (`_`), tetapi mungkin tidak dimulai dengan digit. C adalah case sensitive, jadi `A1` dan `a1` adalah pengidentifikasi yang berbeda. Nama variabel harus dimulai dengan huruf kecil. Kemudian dalam teks, kami akan memberikan signifikansi khusus untuk pengidentifikasi yang dimulai dengan huruf kapital dan pengidentifikasi yang menggunakan semua huruf kapital.

Memilih nama variabel yang bermakna membantu membuat program-pendorong diri, sehingga lebih sedikit komentar yang dibutuhkan. Hindari memulai pengidentifikasi dengan garis bawah (`_`) untuk mencegah konflik dengan pengidentifikasi yang dihasilkan kompiler dan pengidentifikasi perpustakaan standar. Nama variabel multi-kata dapat membuat program lebih mudah dibaca. Untuk nama -nama seperti itu:

- Pisahkan kata -kata dengan garis bawah, seperti dalam `total_commissions`, atau
- Jalankan kata -kata bersama dan mulailah setiap kata berikutnya dengan huruf kapital, seperti dalam `total komisi`.

F. Keywords

Keyword adalah merupakan identifier yang telah didefinisikan oleh bahasa C. Keyword pada bahasa C adalah reserved word yang artinya dicadangkan dan tidak boleh digunakan untuk keperluan lain (nama variabel, nama fungsi (function), nama tipe data, dan lain-lainnya). Semua keyword ditulis dalam huruf kecil.

Beberapa kata yang digunakan dalam contoh, seperti `int`, `if` dan `void`, adalah kata kunci atau kata -kata yang dicadangkan dari bahasa dan memiliki makna khusus untuk kompiler. Tabel berikut berisi kata kunci C. Jangan menggunakannya sebagai identifiers.

Keywords				
auto	do	goto	signed	unsigned
break	double	if	sizeof	void
case	else	int	static	volatile
char	enum	long	struct	while
const	extern	register	switch	
continue	float	return	typedef	
default	for	short	union	
<i>Keywords added in the C99 standard</i>				
_Bool	_Complex	_Imaginary	inline	restrict
<i>Keywords added in the C11 standard</i>				
_Alignas	_Alignof	_Atomic	_Generic	_Noreturn
_Static_assert	_Thread_local			

G. Variable

Semua variabel harus didefinisikan dengan nama dan jenis sebelum dapat digunakan dalam suatu program. Anda dapat menempatkan setiap definisi variabel di mana saja di utama sebelum penggunaan pertama variabel dalam kode. Secara umum, Anda harus mendefinisikan variabel yang dekat dengan penggunaan pertama mereka.

Declaration format:

```
<data type> <variable name>;
```

```
<data type> <variable name> = <initial value>;
```

Contoh:

```
int a, b, c, total;
float salary, bonus;
int num_students = 20;
```

Untuk memasukkan beberapa pernyataan dalam tubuh IF, Anda harus melampirkan pernyataan dalam braces ({dan}). Satu set pernyataan yang terkandung dalam sepasang kawat gigi disebut pernyataan majemuk atau blok. Pernyataan majemuk dapat ditempatkan di mana saja dalam program yang dapat ditempatkan oleh satu pernyataan.

H. Tipe data

Bahasa portabel seperti C harus memiliki ukuran tipe data yang fleksibel. Aplikasi mungkin memerlukan bilangan bulat dari berbagai ukuran. C menyediakan beberapa tipe data untuk mewakili bilangan bulat. Selain int dan char, C memberikan jenis int int (yang dapat disingkat sebagai pendek) dan int int (yang dapat disingkat selama). Ada juga variasi yang tidak ditandatangani dari semua tipe integral yang mewakili nilai integer non-negatif.

Standar C menentukan rentang nilai minimum untuk setiap jenis integer. Kisaran aktual mungkin lebih besar, tergantung pada implementasinya. Untuk ints pendek, kisaran minimum adalah -32767 hingga +32767. Untuk sebagian besar perhitungan bilangan bulat, INT panjang sudah cukup. Kisaran nilai minimum untuk INT panjang adalah -2147483647 hingga +2147483647. Kisaran int lebih besar dari atau sama dengan int pendek dan kurang dari atau sama dengan int int. Di banyak platform saat ini, INTS dan InTS panjang mewakili

kisaran nilai yang sama. Jenis data yang ditandatangani Char dapat mewakili bilangan bulat dalam kisaran -127 hingga $+127$ atau salah satu set karakter ASCII.

Data yang ingin dioleh dengan menggunakan bahasa C harus dapat memiliki tipe yang jelas. Berikut tipe data dasar yang dikenal pada bahasa C :

DATA TYPE	SYNTAX	RANGE
character	unsigned char	0 to 255
	char	-128 to 127
integer	unsigned int	0 to 65535
	int	-32768 to 32767
	short int	-128 to 127
	unsigned long	0 to 4294967295
	long	-2147483648 to 2147483647
float	float	3.4E-38 to 3.4E+38
	double	1.7E-308 to 1.7E+308
	long double	3.4E-4932 to 1.1E+4932

Tipe data dapat digabungkan dengan modifier signed, unsigned, long, dan short untuk menyesuaikan jangkauan nilai yang diinginkan

I. Constant

ANSI C memungkinkan Anda untuk mendeklarasikan konstanta. Ketika Anda mendeklarasikan konstanta itu sedikit seperti deklarasi variabel kecuali nilainya tidak dapat diubah. Konstanta adalah suatu nilai konstan, dimana nilai tersebut tidak berubah nilainya dalam sebuah program.

Kata kunci const adalah mendeklarasikan konstan, seperti yang ditunjukkan di bawah ini:

```
int const a = 1;
```

```
const int a = 2;
```

Catatan:

- Anda dapat mendeklarasikan const sebelum atau setelah jenisnya. Pilih satu stick untuk itu.
- Biasanya untuk menginisialisasi const dengan nilai karena tidak bisa mendapatkan nilai dengan cara lain.

Preprocessor #define adalah metode lain yang lebih fleksibel untuk mendefinisikan konstanta dalam suatu program. Anda sering melihat deklarasi const dalam parameter fungsi. Ini mengatakan hanya bahwa fungsi tidak akan mengubah nilai parameter. Definisi fungsi berikut menggunakan konsep yang belum kami temui (lihat bab tentang fungsi, string, pointer, dan perpustakaan standar) tetapi untuk penyelesaian bagian ini termasuk di sini:

```
void strcpy(char *buffer, char const *string)
```

String argumen kedua adalah string C yang tidak akan diubah oleh string Menyalin fungsi pustaka standar. Konstanta yang dikenal pada bahasa C yaitu :

Type Constant:

- ① *Integer constant* ⇒ -5
 - ② *Floating-point constant* ⇒ 3.14
 - ③ *Character constant* ⇒ 'C' '1' '\$'
 - ④ *Escape sequence* ⇒ \n \t \"
 - ⑤ *String constant* ⇒ "BiNus"
 - ⑤ *Symbolic constant* ⇒ #define PHI 3.14
 ⇒ const float PHI=3.14;
- ☞ 'H' is a *character constant*
 - ☞ "H" is a *string constant*
 - ☞ 1 is a *integer constant*
 - ☞ '1' is a *character constant*
 - ☞ const float Pi= 3.1415926; Pi adalah konstanta simbolis konstan

J. Sizeof

Sizeof operator biasanya merupakan operasi waktu kompilasi, tetapi beroperasi saat runtime saat diterapkan pada VLA (Variable-length array). Jendela output menunjukkan bahwa sizeof operator mengembalikan ukuran 24 byte - empat kali angka yang kami masukkan karena ukuran int pada mesin kami adalah 4 byte.

Contoh :

```
#include <stdio.h>
int main()
{
    printf("Size int : %d", sizeof(int));

    getchar();
    return 0;
}
```

Size int : 4_

Dari contoh tersebut dihasilkan bahwa ukuran dari tipe data int besarnya adalah 4 byte.

K. Suffix

Sufiks integer dan floating-point memungkinkan Anda untuk menentukan secara eksplisit tipe data nilai literal. Secara default, tipe integer literal ditentukan oleh tipe pertama yang mampu menyimpan nilainya- int, unsigned long int, dll. Sebuah titik mengambang literal tanpa akhiran memiliki tipe ganda.

Sufiks integer adalah U atau u untuk ints unsigned, L atau l untuk ints panjang, dan ll atau ll untuk ints yang panjang. L dan LL lebih disukai untuk keterbacaan, karena huruf kecil L dapat disalahartikan sebagai 1 (satu). Anda dapat menggabungkan u atau U dengan int int dan panjang panjang untuk membuat literal yang tidak ditandatangani untuk tipe integer yang lebih besar. Literal berikut memiliki jenis unsigned int, long int, unsigned long int, dan unsigned long long int:

```
174u
8358L
28373ul
9876543210llu
```

Sufiks floating-point adalah F atau F untuk pelampung dan L atau L untuk ganda panjang. Sekali lagi, L lebih disukai untuk keterbacaan. Berikut ini adalah melayang dan literal ganda panjang:

1.28f

3.14159L

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Bahasa C disebut bahasa pemrograman struktural karena memiliki pola main function, modularisasi dengan fungsi, serta kontrol alur yang jelas. Diskusikan: mengapa pola struktural ini membuat bahasa C cocok digunakan untuk membuat sistem operasi dan program berskala besar?

Hint:

- Bahas peran fungsi **main()** sebagai entry point.
- Modularity: bisa pecah program jadi fungsi-fungsi kecil → code reusable dan maintainable.
- Koneksi ke OS: bahasa C dekat dengan hardware tapi tetap readable → cocok untuk driver, kernel, compiler.

Pertanyaan Diskusi:

Bahasa C sangat sensitif terhadap huruf besar-kecil (case sensitive) dan penulisan syntax (contoh: titik koma wajib). Bagaimana dampaknya jika programmer ceroboh dengan case dan tanda baca dalam coding? Berikan contoh error yang mungkin muncul!

Hint:

- Bahas contoh `printf` vs `Printf` → error: *undefined reference*.
- Contoh lupa tanda `;` → syntax error saat compile.
- Diskusikan: di C compiler sangat ketat → bagaimana cara mencegah typo? (IDE, code editor dengan linting, debugging).

Pertanyaan Diskusi:

Dalam Bahasa C, penamaan identifier (variabel, fungsi) dan penggunaan constant sangat diatur. Diskusikan: mengapa memilih nama variabel yang bermakna & mendeklarasikan constant penting untuk menjaga kualitas program?

Hint:

- Bandingkan: variabel `a` vs `total_price` → mana lebih jelas?
- Diskusi: jika constant dideklarasikan dengan `#define` atau `const`, lebih aman untuk maintenance → menghindari hard-coded value.
- Bahas gaya penulisan: camelCase vs snake_case → membantu readability tim.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Struktur Dasar Pemrograman Bahasa C: [YouTube](#))

Post-Test



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

1. Pernyataan manakah yang TIDAK termasuk karakteristik algoritma yang benar?

- A. Memiliki input yang terdefinisi dengan baik
- B. Harus memiliki output yang tidak terbatas
- C. Langkah-langkahnya harus jelas dan tidak ambigu
- D. Harus selesai dalam jumlah langkah terbatas

2. Pada tahap representasi algoritma, tahap pseudo-code digunakan untuk ...

- A. Menyimpan hasil output ke file
- B. Menyusun instruksi algoritma dengan bahasa manusia dan notasi matematika
- C. Menulis program dalam bahasa mesin
- D. Membuat diagram alur secara grafis

3. Berikut ini manakah yang merupakan contoh struktur selection dalam bahasa C?

- A. `for(i = 0; i < 10; i++)`
- B. `if(score > 70) printf("Lulus");`
- C. `total = a + b + c;`
- D. `printf("Hello World");`

4. Pernyataan mana yang BENAR terkait sensitivitas huruf (case sensitive) pada bahasa C?

- A. `printf` dan `Printf` dianggap sama oleh compiler
- B. Setiap pernyataan harus diakhiri tanda koma (,)
- C. Variabel `Total` dan `total` dianggap sama
- D. Fungsi `main()` wajib ada sebagai fungsi utama program C

5. Fungsi `printf()` tidak akan dikenali jika kita lupa menuliskan ... di awal program C:

- A. `#define`
- B. `#include <stdio.h>`
- C. `#include <math.h>`
- D. `return 0;`

Kesimpulan

1. *Standart library* C merupakan satu set fungsi, konstanta, file header yang dapat digunakan untuk membuat program dengan bahasa pemrograman C.
2. Bahasa C merupakan bahasa pemrograman yang struktural hal ini dikarenakan:
 - Bahasa pemrograman C hanya memiliki 1 fungsi utama (fungsi *main*)
 - Bahasa pemrograman C sangat *case sensitif*.
 - Antara *statement* 1 dan *statement* 2 terlihat perbedaannya yaitu ketika fungsi **printf** dengan **Printf** maka **statement** 2 akan tidak dikenali dalam bahasa C.
 - Setiap pernyataan pada bahasa C harus diakhiri dengan; (*semi-colon*).
3. Komen di bahasa pegraman C digunakan untuk perintah yang tidak akan dijalankan ketika program tersebut di *compile*.
4. Escape sequence digunakan untuk mewakili karakter khusus atau tertentu dalam *string* literal dan karakter literal.
5. Karakter, Identifier, Keyword in C
 - Karakter pada bahasa C
Bahasa program C ditulis dengan menggunakan sebagian dari karakter ASCII (*American Standard Code for Information Interchange*).
 - Identifier pada bahasa C
Identifier adalah nama berbagai elemen program seperti : nama variabel, nama fungsi (*function*), nama tipe data, dan lain-lainnya.
 - Keyword pada bahasa C
Keyword adalah merupakan identifier yang telah didefinisikan oleh bahasa C. *Keyword* pada bahasa C adalah reserved word yang artinya dicadangkan dan tidak boleh digunakan untuk keperluan lain (nama variabel, nama fungsi (*function*), nama tipe data, dan lain-lainnya).
6. Variabel adalah tempat yang digunakan untuk menampung data. Jenis data yang dapat ditampung pada suatu variabel harus ditentukan terlebih dahulu tipe datanya.
7. Data yang ingin diolah dengan menggunakan bahasa C harus dapat memiliki tipe yang jelas.
8. Konstanta adalah suatu nilai konstan, dimana nilai tersebut tidak berubah nilainya dalam sebuah program

Daftar Pustaka

- Paul Deitel & Harvey Deitel. (2022). C how to program. 09. Pearson Education. Hoboken. ISBN: 978-0-13-739839-3. Chapter 2, 3 , 4, 5 and 9 .
- https://www.tutorialspoint.com/data_structures_algorithms/algorithms_basics.htm
- Writing Your First C Program: <http://aelinik.free.fr/c/ch02.htm>
- Data Types and Names in C: <http://aelinik.free.fr/c/ch04.htm>
- Programming in C: [http:// www.cs.cf.ac.uk/Dave/C/](http://www.cs.cf.ac.uk/Dave/C/)
- Pseudocode Examples: <http://www.unf.edu/~broggio/cop2221/2221pseu.htm>
- C Language Tutorial: <http://www.lysator.liu.se/c/bwk-tutor.html>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press.
- Tarsini, I., & Anggraeni, R. (2024). Explore flowchart and pseudocode concepts in algorithms and programming. Indonesian Journal of Multidisciplinary Science, 3(5).

Berikut jawaban untuk pertanyaan Post-Test diatas:

1. **Jawaban: B**

Alasan:

Karakteristik algoritma mengharuskan output terdefinisi dengan baik & terbatas, bukan tidak terbatas.

2. **Jawaban: B**

Alasan:

Pseudo-code adalah representasi algoritma dalam bentuk semi formal yang mudah dibaca manusia sebelum dikonversi ke source code.

3. **Jawaban: B**

Alasan:

if adalah contoh struktur **selection**, yaitu seleksi kondisi yang dieksekusi jika syarat terpenuhi.

4. **Jawaban: D**

Alasan:

Bahasa C mewajibkan fungsi `main()` sebagai entry point. Selain itu, `printf` $\neq$ `Printf`, dan case sensitive membedakan `Total` dan `total`.

5. **Jawaban: B**

Alasan:

`printf()` adalah bagian dari standard library I/O `<stdio.h>`. Tanpa `#include <stdio.h>`, fungsi tidak dikenali.